Pre-submission validation research

Observation-Guided Combination Testing for Coding-Agent Patches

Program analysis + contract-based semantic reasoning

16/17 development trials detected

Unified abstraction still under development

Research progress report

1 October 2026

Score: completed fresh v3 run; 14 distinct tasks. Detection is not issue resolution.

The problem: a patch passes its validations but remains wrong

Available before submission

- Issue and candidate patch.
- Repository and full original trajectory.
- Tests, scripts, failures and fixes.

Replay the historical state to simulate a plugin before submission.

Effective validation gap

Validations exercise individual behaviors but miss an interaction, or use an unsuitable observation or expectation.

Goal: add or modify a valid combination test that exposes the submitted patch's defect.

Intended endpoint: test feedback → patch repair → hidden-test evaluation. This repair loop is not yet evaluated.

Setting: DeepSWE replay; original trajectory is allowed input. Hidden tests and reference patches are evaluation-only.

Real DeepSWE failure: HTTPX streaming JSON (N01)

Issue: incremental JSON iteration must reject malformed data and close a consumed streaming response.

Validation context	Valid JSON	Invalid JSON
In-memory response	Tested	Tested
Live response stream	Tested	Missing close check

Normal streaming tests checked consumed/closed state; malformed-line tests used in-memory content.

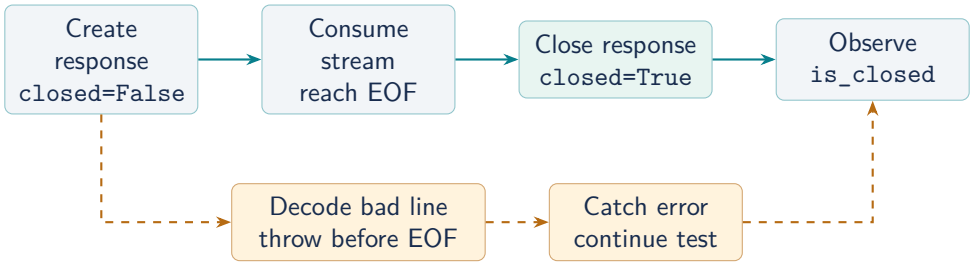
What escaped validation?

A complete malformed NDJSON line is decoded **before the stream reaches EOF**. Parsing throws; normal close is skipped.

Original agent: `deepseek-v4-flash`. The trajectory records **156 passing tests** after incremental parsing was introduced.

Public N01 trial; trajectory steps 57, 77–82, 95. Matrix summarizes relevant checks, not all tests.

N01: the error must occur before the write that closes the stream



The same response and the observation after the exception matter. A malformed buffered JSON response may already be closed and reveal no defect.

N01 candidate behavior and original trajectory. Arrows illustrate execution order; they do not claim a complete static proof.

Four challenges determine whether a combination is useful

Find the useful validation

Many temporary tests and variants compete for a small review budget. Historical tests may also become stale after later edits.

Make interaction observable

Inputs must change what the assertion can see. One-field schemas or normal completion can hide the interaction.

Connect the right context

Operations must concern the right object, input source and observation stage. Co-occurrence alone is insufficient.

Keep a correct oracle

A combined context can invalidate old expectations. The submitted implementation may be wrong, so its output is not ground truth.

Observed across 17 development trials: N01 exception timing; N09 partial defaults; N05 oracle interpretation; N06 historical validation recovery.

Three observations: two tested operations do not imply a tested interaction

The relation can change

The same consume/error/check sequence may throw before or after closing a response.

N01, HTTPX: error timing changes which write the close assertion reads.

Object and stage matter

A boundary observed at initialization does not test that boundary after a nonempty batch.

N08, Pebble: wait on the same database at the exclusive batch end.

Combination may be invisible

Supplied parameters can be correct while omitted default fields are wrong.

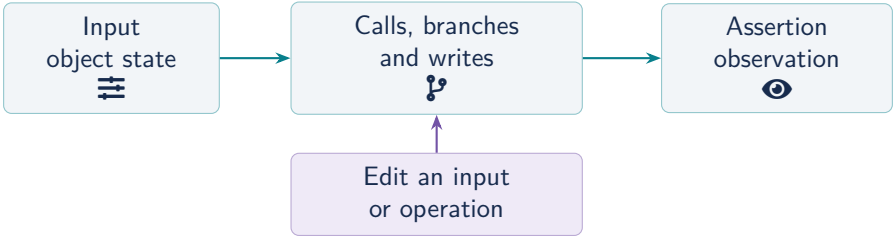
N09, Koota: check omitted fields in a partial record before flush.

Use one concrete observation to connect the input, object history and execution stage.

Empirical findings from the development trajectories. They motivate the abstraction; they are not a global novelty or completeness claim.

Core abstraction: the production path of an observation

Unit: an assertion on a specific object at a specific execution stage, together with the operations producing its observed value.



Search question
Can an edit change how this state is produced, while keeping the observation reachable?

Semantic question
What should hold in the new context, according to the issue and API contract?

Research abstraction; bounded pieces are implemented. It is not a complete dependence model or a unified composition algorithm.

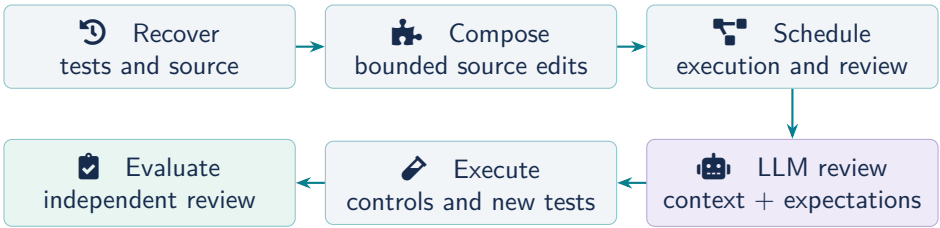
Why an observation path helps expose a missing interaction

Executable context	Visible interaction	Contract reasoning
Reuse source inputs and calls; align their arguments and receiver objects.	Trace the operations producing the observed value, including its stage.	Provide relevant source facts; decide whether every expectation still holds.
N01: malformed bytes enter the same live response whose close state is checked.	**N09:** read the omitted default fields in the pending record, before flush.	**Both:** existing close/default expectations remain correct and expose the defect.

Analysis organizes the relevant program facts; the model reasons about a concrete combination and its complete expected behavior.

This explains the mechanism in two confirmed examples. It is not a completeness or generalization guarantee.

Overall workflow: source structure narrows semantic reasoning



Two execution routes: keep source expectations; also execute accepted LLM revisions.

Iteration: preserve unserved candidates; expand the source frontier.

Next: use valid failures for patch repair and hidden-test evaluation.

Scored v3 uses existing writer/impact scheduling. The newer patch-to-observation ranking is a separate optional prototype.

How existing tests become executable combination candidates

Parse tests and assertion APIs from repository and trajectory artifacts. Preserve their definitions, calls, fixtures and imports.

Source-derived building blocks

- Inputs and argument positions.
- Calls, bindings and object reuse.
- Assertions and required execution prefixes.

Finite edits: parameter replacement, operation composition, shared-object binding and field-presence changes.

N01: reuse two existing contexts

Streaming test:
live response + close observation

Error test:
malformed NDJSON + decoding-error check

Combination:
malformed input in the live response; observe close after catching the error.

Executability does not establish semantic correctness. Deriving new edits directly from paths remains open.

The LLM gets one concrete semantic task

Program-produced evidence

- Fixed input/operation edits.
- Complete test and all assertions.
- Calls, arguments and source bodies.
- Issue contract and analysis unknowns.

Semantic decision

Is this a legal context? Which expectations remain valid? What should replace the expectations that change?

Output: retain, revise, or unknown. Code applies supported edits to expectations, preserving actions and observations.

Infer expectations from the contract, not the buggy output. Review all assertions and prerequisites.

Completed v3: at most 16 semantic candidates per trial per round; one full candidate per request; fresh DeepSeek-v4-flash requests.

Formal problem: find a valid defect-exposing combined context

Input: candidate program P, issue contract I, repository R, original trajectory H, and budgets B.

$$T_0 = \mathsf{Extract}(R, H), \qquad \mathcal{C} = \{\mathsf{Apply}(t, \delta) \mid t \in T_0,\ \delta \in \mathcal{E}(T_0, R)\}.$$

T_0: recovered tests; $\mathcal{E}$: finite input/operation edits; $\mathcal{C}$: combined contexts before oracle review.

Search objective

Find $c \in \mathcal{C}$ and reviewed expectations o. Materialize $u = \mathsf{Test}(c, o)$ such that:

$$\mathsf{Valid}_I(u) \ \wedge \bigwedge_{s \in \mathsf{Controls}(u)} \mathsf{Pass}(P, s) \ \wedge \ \mathsf{IssueFail}_I(P, u).$$

Valid: legal test and correct complete expectations. **Controls**: associated source tests. **IssueFail**: confirmed issue-related failure, including a defect-induced panic.

Independent semantic review; compile/environment errors excluded. Budgets limit composition, requests and execution.

Formal success criterion: a valid, issue-related native failure

$\mathrm{Valid}_I(u)$: semantic validity

Legal setup, meaningful observations and correct complete expectations, including assertions before the final observation.

$\mathrm{Pass}(P, s)$: source control

Associated original tests pass on the same replayed candidate program.

$\mathrm{IssueFail}_I(P, u)$: defect evidence

Confirmed issue-related native test failure: an assertion mismatch or a defect-induced exception/panic under valid setup.

Compile and environment errors do not qualify. Semantic validity is assessed independently; our analyzer does not prove it.

The defect escaped effective pre-submission validation. Its combined relation need not be absent from every earlier trajectory step.

N06 recovers a historical combination not revalidated on the final patch. P5 exposes a defect-induced VM panic before later assertions.

Formal abstraction: an observation and its production path

For assertion q, keep its object binding r_q, source execution stage τ_q, and supported backward dependence slice Π_q:

$$O_q = (q, r_q, \tau_q, \Pi_q), \qquad \Pi_q = \mathrm{Slice}(a_q, G),$$

a_q: actual-value expression; G: bounded source graph. Edges record value, control, calls and supported field effects.

Candidate relevance

An edit d is relevant when $d \rightsquigarrow a_q$ in the supported graph. Object binding and source position distinguish two similar observations.

A concrete state-path change

N01 reads the same close field after consumption. Its last observed writer changes from `close` to `constructor` when parsing throws early.

Research abstraction, partly implemented. Scored impact uses the original local graph; prompt evidence reparses edited code. Last-writer feedback is an independent prototype.

Formal method: compose, budget semantic review, then execute

Let T be recovered source tests; F_k the search frontier after round k; and A the catalogue of unserved semantic candidates.

	Operation	Meaning
1	$C_k \leftarrow \text{Compose}(T, F_{k-1})$	Apply bounded, source-derived input and operation edits.
2	$A \leftarrow A \cup C_k$	Preserve candidates awaiting semantic review.
3	$Q_k \leftarrow \text{SelectImpact}(A, B_o)$	Choose a bounded set using assertion dependencies.
4	$V_k \leftarrow \text{Review}(I, Q_k)$	LLM retains/revises complete expectations, or returns unknown.
5	$F_k \leftarrow \text{Execute}(P, C_k \cup V_k)$	Run original-oracle and revised tests with source controls; update feedback.

Formal selection: spend review on potentially stale expectations

For candidate u and unchanged assertion q, let D_u be edited source nodes, a_q its actual expression, and e_q its expected expression.

$$\mathrm{Impact}(u) = \{(q, d) \mid d \in D_u,\ d \rightsquigarrow a_q,\ d \not\rightsquigarrow_{\mathrm{value}} e_q\}.$$

Interpretation

The actual result may depend on the edit, but the expected value does not adapt with it. The old expectation may need review.

Deterministic allocation

Greedily cover new assertions, then new assertion–edit pairs. Every fourth eligible rank uses the original family/seed order.

Eligibility requires an expectation that the current interface can edit. Unsupported dependencies retain exploration opportunities.

Scored v3 impact policy: original local may graph, not a feasibility/oracle proof. This query also accepts simple mutations; it does not itself prove a missing interaction.

A path-based rule already helps select useful combinations

Implemented Python rule: prioritize a candidate with all four source conditions below.

Preserve the observation

Keep the original assertion after a newly introduced exception handler.

Find a possible bypass

Source calls contain an exception exit before the write producing the observed field.

Align object bindings

The handled operation and the observed field concern the same supported receiver binding.

Allocate execution first

Move supported candidates ahead of the fallback order; retain other candidates.

N01: a malformed live stream can bypass close. Execute to check timing; use the issue contract to check the oracle.

Bounded source rule, not an exception-feasibility proof. Equal-budget Python experiment: 3/8 to 4/8 confirmed detections; no LLM calls.

N01 worked example: the combined test checks close after failure

```python
def chunks():
    yield b'{"a": 1}\nnot json\n'
    yield b'1}'  # unread when parsing fails

response = httpx.Response(
    200,
    content=chunks(),
    headers={"Content-Type":
             "application/ndjson"},
)
with pytest.raises(httpx.DecodingError):
    list(response.iter_json())
assert response.is_closed
```

Why this input works

A malformed complete line is parsed while the stream is live. Catch the error, then observe close.

Contract: close after streaming consumption.

Required: `True`.
Candidate returns: `False`.

The original close oracle is retained.

Executed witness condensed: imports, names and additional checks omitted here.

Executed N01 witness c8e075c82686eb21c4222575; source and native execution in the v3 ledger.

N01 worked example: evidence from recovery to native failure

Stage	What actually happened in the completed v3 run
Recover	Stream construction and close checks; malformed donor input and error handling.
Compose	Align the response; transfer input, header and exception context into its live generator.
Select	Selected for execution; source close oracle retained. This witness required no LLM revision.
Execute	Source controls pass. Generated pytest test fails: `is_closed` is `False`.
Confirm	Independent evaluation checks the complete oracle, emitted source and native failure.

A complete native defect witness on the replayed submitted patch; the patch itself is not repaired.

N01 deliveries and runtime index; main v3 `checkpoint_14.json`.

Second real success: Koota partial defaults before flush (N09)

Contract: deferred `get` must return the same logical values as after `flush`. A partial parameter object must retain omitted defaults.

```
// Existing Test fixture defaults include:
// current=1, test='hello', bool=true.
const entity = world.spawn();
world.deferred.add(entity,
  Test({current: 2, arr: ['d','e','f']}));
const test = entity.get(Test); // no flush
expect(test).toMatchObject({
  current: 2, test: 'hello', bool: true,
  arr: ['d','e','f']
});
```

Condensed executed witness; the complete original fixture, class/default checks and later actions are preserved in the actual test.

Recover: repository test for ordinary partial `entity.add`.

Compose: replace that call with `world.deferred.add`; preserve bindings and expectations.

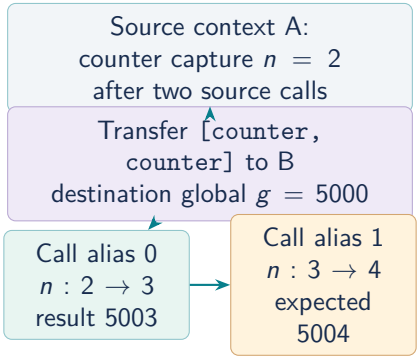
Observe: read before flush. The buggy projection returns supplied parameters without omitted defaults.

✅ Source control passes; generated Vitest test fails on missing defaults.

Public N09 trial; main witness `08f2c625387570b30312`. Source expectations retained.

Where the LLM matters: shared closure aliases in Tengo (P6)

Contract: transferred callables use destination globals and preserve shared captured state, including inside an array.



One concrete semantic revision

Source expected **103** belonged to a later call in A. The combination calls the second alias in B instead.

Program fixes the input/actions. Fresh LLM review changes only the final expected value: **103** $\to$ **5004**.

Native result: 5003. The transferred aliases incorrectly have independent captures. Source control passes; the complete revised oracle is independently

Evaluation protocol: a confirmed witness, not any failing test

Completed main experiment

- 17 trials, 14 distinct tasks.
- 7 original pilots + 10 Flash failures.
- All used during development.
- Frozen v3: `7a8ff95`; 2 rounds.

Fresh oracle requests; original trajectories available. No prior-answer reuse.

Trial-level detection metric

Count a trial only with legal context, correct complete expectations, passing controls and a named issue-related failure.

Separate metrics: target exposure/rank, complete oracle correctness, execution errors, calls and tokens.

Rank, test correctness and issue resolution are different metrics. Overall false-positive rate is not yet established.

v3 independent evaluation, `checkpoint_14.json`. P1–P3 share a task; P5/P6 share another.

Status of the original pilots: 7/7 confirmed defect witnesses

A hit requires a valid complete test, passing source controls and an issue-related native failure on the submitted patch.

Case	Project	Interaction exposed	Hit	Oracle
P1	Textual	Input encoding family × event phase	✓	Revised
P2	Textual	Shortcut modifier × nonempty text	✓	Retained
P3	Textual	Shift modifier × shifted-key metadata	✓	Retained
P4	IPython	Quoted/escaped representation of a file path	✓	Retained
P5	Tengo	Returned callable × container traversal	✓	Retained
P6	Tengo	Shared capture × recursive callable transfer	✓	Revised
P7	Valibot	Recursive schema × runtime transform	✓	Revised

Retained = source expectations remain valid; revised = fresh model expectations independently confirmed. Pilots

Status of ten additional Flash trials: 9/10 confirmed witnesses

All ten original agents used `deepseek-v4-flash`. These are development trials selected from failed DeepSWE runs.

Case	Project	Interaction exposed	Hit	Oracle
N01	HTTPX	Streaming JSON error × close	✓	Retained
N02	HTTPX	Multipart parsing error × close	✓	Retained
N03	mnamer	Duplicate normalized path × quota	✓	Retained
N04	sqlite-utils	Bulk UPDATE × invariant-read field	✓	Revised
N05	termenv	Fitting width × preserved ANSI reset	!	Miss
N06	etree	Sibling insertion × descendant text	✓	Retained
N07	go-git	Overlapping edits from the same base	✓	Retained
N08	Pebble	Completed batch × exclusive frontier	✓	Retained
N09	Koota	Deferred partial parameters × defaults	✓	Retained
N10	DynamoDB	Omitted mode × conditional requiredness	✓	Retained

Results: practical progress with limited causal evidence

Completed main run

16/17

Confirmed development-trial witnesses



540 fresh calls; 7.42M total tokens.
83 responses reached the output limit.

Isolated local evidence

Same eight Python pools, eight proposals each, no LLM: writer-path static priority improved confirmed detection **3/8** → **4/8**.

Added N01. This supports a local search rule, not a universal abstraction.

Earlier 12/17 and current 16/17 use different configurations; their difference does not isolate the analysis contribution.

Main score: v3 checkpoint 14 and provider receipts. Local equal-budget experiment: assertion-writer search report. Tokens include reasoning once.

Academic contribution: an insight to turn into a general algorithm

Research hypothesis: use one observation's production path to derive and prioritize combinations; use contract reasoning to decide their expectations.

Design element	Current academic assessment
Empirical insight	Operation co-occurrence is insufficient: object, stage and what the assertion reads determine effective validation.
Supported mechanism	Exception/writer selection exposes N01 at equal budget: 3/8 $\rightarrow$ 4/8 in the local Python comparison.
Existing techniques	Slicing, binding, call graphs and analysis-assisted LLM reasoning are established; combining them is not itself novel.
Contribution still open	A general path-driven synthesis operator, with equal-budget gains on unseen tasks. Current constructors and negative ranking results do not establish it.

Hypothesis to test: observation-driven search improves valid defect detection at equal budget.

Current limitations: selection, oracle reliability and analysis cost

N05: an unresolved complete witness

Fitting width + preserve reset. Target ranks **96 and 122** in the two rounds; never offered. A related executed variant kept an invalid truncation expectation.

Both budget allocation and complete oracle inference remain bottlenecks.

Analysis coverage and cost

The later strict path study supports **0/4,502** candidates. One 370-candidate Go pool took **33.2 minutes**; heap, helper and callback support is incomplete.

New source-only caching reduces sampled analysis cost, while keeping evidence unchanged. It adds no detection result.

34 sampled raw failures: 30 invalid tests, 2 valid detections, 2 uncertain contracts. This is not the overall false-positive rate.

N05 final audit; fixed SHA failure sample; patch-priority v1 census. Later cache study is separate and detailed in the appendix. Overall false-positive rate remains unmeasured.

Next research steps: make the abstraction drive synthesis

Path-driven composition

Derive new edits from paths, beyond current constructors. Compare under equal budgets.

Bounded analysis cost

Index assertions and cache call facts. Measure whole-pool and online latency.

Complete observations

Preserve object identity, phases and prerequisites. Audit every assertion against contract evidence.

Independent evaluation

Freeze unseen trials; repeat model runs; evaluate repair with hidden tests.

Success: more valid witnesses and better repaired patches at fixed total budget, with measured misses and false alarms.

Planned work. Current evidence establishes local detection progress, not zero false negatives, generality, or issue-resolution improvement.

Later prototype: candidate-specific edit–patch–observation paths

Reparse edited test u. Let D_u be its edited value nodes, Δ the submitted patch's changed production nodes, and a_q an actual observation.

$$d \in D_u \;\rightsquigarrow\; p \in \Delta \;\rightsquigarrow\; a_q.$$

Join parameter bindings at the same source call site; do not transfer a source test's score to every edit.

Two optional predicates

$S(u)$: supported return-dependence projection connects the edit, changed code and actual value.

$M(u)$: the edit affects a call in the actual-value slice, and a callee uniquely resolved by the supported source rules can reach changed code. Weaker may evidence.

$$\mathrm{Rank}_m(F) = [u \in F \mid m(u)] \parallel [u \in F \mid \neg m(u)], \quad m \in \{S, M\}.$$

F: fixed family/seed/shape order; $\parallel$: stable concatenation. Unknown candidates remain in fallback.

Latest ranking prototype: distinguish edits inside the same test

```
# Submitted patch changed this return.
def calculate(value, unused):
    return value * 3

result = calculate(1, 10)
assert result == 3
```

Candidate A: change 1 to 2.
Candidate B: change 10 to 11.

Candidate-conditioned dependency

Both candidates call the modified function. Only A's edited argument participates in the return value seen by the assertion.

Strict: project arguments through finite return dependencies.

May: use a conservative same-call path to a modified callee; may include unused arguments.

Reparse the edited test and join at the same call site. A missing path remains unknown.

Illustrative fixture; not a DeepSWE result. Completed real-pool study: strict finds no priorities; may reduces witness exposure.

Completed ranking study: no improvement at equal static budget

Fixed 4,502 candidates; 17 development trials / 14 tasks; budget 16 per trial. Rank rules frozen before witness matching.

Metric	Shape	Strict	May
Raw witness-program exposure@16	9/17	9/17	7/17
Common legacy eligible exposure@16	7/17	7/17	5/17
Ranking time, whole pools (minutes)	0.81	75.80	75.73

Strict supports 0/4,502 paths; may prioritizes 925. May loses N02 and P5. N05 rank: 170 $\rightarrow$ 111, still outside budget.

Conclusion: return-only support is too narrow; may reachability is too coarse. Retain the previous main configuration.

Exposure: a known witness input program appears in the first 16. Eligible: common old assertion interface. N08 absent from pool. No new model/native runs; main 16/17 unchanged. See `research/patch_observation_priority_results.md`.

Latest source-only efficiency study: cache without changing evidence

Freeze 5 candidates per trial by deterministic hash order: 85 candidates, 17 trials. Run three fresh-process analysis arms; compare complete evidence with sealed v1.

	Frozen v1	Lexical cache	+ Assertion index
Exact evidence equality	85/85	85/85	85/85
Total analysis time (seconds)	100.26	65.42	58.07
Supported strict / may	0 / 21	0 / 21	0 / 21

Aggregate speedup: 1.73×; median trial speedup: 1.07×. Most savings came from P6; N08 did not accelerate. Index construction is included.

An implementation cost improvement; source support and native detection are unchanged. No model or repository tests were run.

17/17 trials and 51/51 analysis workers complete. Separate study: `results/patch_priority_cache_equivalence_20261001/run_v1/summary.json`. Sample cost is not full-pool

Reference points for the research design

- **STARS, ISSTA 2026**: static analysis synthesizes assertion components, including type-connected observer call chains; LLMs concretize assertions.
- **Change And Cover, ICSE 2026**: patch-focused regression-test augmentation and recovery of project-specific test context; assumes a bug-free PR.
- **Testora, ICSE 2026**: compares pre/post-change behavior and uses documented change intent to classify differences.
- **Mixed-Assertion Scenario, ISSTA 2026**: multiple assertions jointly express test objectives; known insertion positions are a consequential assumption.

Our comparison must isolate candidate composition, observation relevance and oracle correctness. Classical analysis and context retrieval are strong baselines.

Primary author papers and official conference abstract. These comparisons motivate design choices; they do not establish global novelty.

Evidence map and reproducible scope

Material	Repository-relative location
Current method	`research/method_design.md`
Original trials	`research/expanded_trials_20260926/cases.json`; `data/relational_gap_v1/N01/trajectory.json`; N09 likewise
Main score	`results/relation_search_20260929_v3/evaluation/uniform_fresh_impact_v3/checkpoint_14.json`
Native witnesses	Same directory: N01 deliveries and N09 runtime index P6 oracle runtime index; emitted group 017 and request 002
Implementation	`relation_search/`; `scripts/run_relation_campaign.py`
New ranking	`experimental/patch_observation_priority.py`
Cache study	`results/patch_priority_cache_equivalence_20261001/`
This presentation	`docs/combination_testing_research.tex`

Frozen distinction: scored v3 is `7a8ff95`; later source-priority integration is `7615f4e`. New prototype results do not overwrite the scored run.